

Student Result Prediction System

A Machine Learning Web Application using Python & Streamlit

Project Title	Student Result Prediction System
Technology	Python, Streamlit, scikit-learn, Pandas, NumPy, Matplotlib, Seaborn
Domain	Machine Learning / Data Science
Model Used	Logistic Regression
Dataset	500 Synthetic Student Records
Developed By	A.Navaneetha
Date & Version	June 2026 , 1.0

1. Abstract

The Student Result Prediction System is a complete Machine Learning web application developed using Python and Streamlit. It predicts whether a student will PASS or FAIL based on five key academic and lifestyle parameters. The application uses Logistic Regression trained on a synthetic dataset of 500 students and delivers real-time predictions with confidence scores, interactive data visualisations, and model performance metrics.

This project demonstrates the full Data Science pipeline — from synthetic data generation and preprocessing, through model training and evaluation, to deployment as an interactive web application — without requiring any HTML or JavaScript.

2. Objectives

- Build a real-world ML-powered web application from scratch using Python
- Train a Logistic Regression classifier on a realistic synthetic student dataset
- Provide instant PASS / FAIL predictions with confidence percentages
- Visualise data insights using Matplotlib and Seaborn graphs
- Display model performance metrics: Accuracy, Precision, Recall, F1 Score, and Confusion Matrix
- Demonstrate the complete Data Science pipeline from data to deployment
- Create a clean, interactive Streamlit UI usable without any ML background

3. Problem Statement

In educational institutions, identifying at-risk students early can help teachers and administrators take corrective action before final examinations. Traditional methods rely on manual observation, which is slow, subjective, and error-prone.

This project automates the identification process using a trained Machine Learning model that analyses multiple student parameters simultaneously and delivers an instant, data-driven prediction. The system helps educators intervene proactively for students showing early signs of academic difficulty.

4. Tools & Technologies Used

Tool / Library	Version	Purpose
Python	3.10+	Core programming language
Streamlit	1.35+	Web application framework — converts Python into a live website
scikit-learn	1.4+	ML library — Logistic Regression, train/test split, metrics
Pandas	2.0+	Data manipulation and DataFrame operations
NumPy	1.26+	Numerical computations and synthetic data generation
Matplotlib	3.8+	Scatter plots, histograms, bar charts
Seaborn	0.13+	Confusion matrix heatmap visualisation

4.1 Installation Command

```
pip install streamlit scikit-learn pandas numpy matplotlib seaborn
```

5. Dataset Description

A synthetic dataset of 500 student records was generated using NumPy random functions. Realistic value ranges were chosen to simulate real-world academic scenarios.

Feature	Range	Description
Hours_Study	1 – 12	Daily study hours per day
Attendance	40 – 100%	Class attendance percentage

Feature	Range	Description
Previous_Marks	30 – 100	Marks obtained in the previous exam
Assignments_Done	0 – 10	Number of assignments submitted
Sleep_Hours	4 – 10	Average daily sleep hours
Result (Target)	0 / 1	0 = Fail (marks < 50), 1 = Pass (marks >= 50)

5.1 Marks Formula

The final marks for each student are computed using a weighted formula:

```
marks = (0.30 x Previous_Marks)
        + (0.25 x (Hours_Study / 12 x 100))
        + (0.20 x Attendance)
        + (0.15 x (Assignments_Done / 10 x 100))
        + (0.10 x (Sleep_Hours / 10 x 100))
        + random_noise (mean=0, std=5)
```

Marks are clipped to the range [0, 100]. A student is labelled Pass (1) if marks >= 50, otherwise Fail (0).

6. System Architecture

The application follows a single-tier pipeline where all components — data generation, preprocessing, model training, prediction, and visualisation — run within the same Python process served by Streamlit.

Step	Component	Description
1	Data Generation	NumPy generates 500 synthetic student records with realistic parameters
2	Pre-processing	Pandas organises data into a DataFrame; StandardScaler normalises features
3	Model Training	80/20 train-test split; Logistic Regression fitted on training data
4	Prediction	User input is scaled and passed through model.predict() for PASS/FAIL verdict
5	Visualisation	Matplotlib and Seaborn charts are rendered inside Streamlit tabs

Step	Component	Description
6	Web UI	Streamlit serves the entire app on localhost:8501 with no HTML/JS needed

7. Application Features

7.1 Tab 1 — Predict Result

- User adjusts 5 sliders: Study Hours, Attendance, Previous Marks, Assignments Done, Sleep Hours
- On clicking Predict, the model returns PASS or FAIL instantly
- Confidence percentage is shown alongside the prediction badge
- A probability breakdown bar chart displays PASS vs FAIL probabilities
- An input summary table shows the values entered by the user

7.2 Tab 2 — Data Analysis

- Scatter plot: Study Hours vs Marks (colour-coded by Pass/Fail)
- Scatter plot: Attendance vs Marks (colour-coded by Pass/Fail)
- Pie chart: Overall Pass / Fail distribution in the dataset
- Histogram: Marks distribution for Pass and Fail students
- Summary statistics: Total students, Pass count, Fail count, Pass rate %
- Expandable section showing the first 20 rows of raw data

7.3 Tab 3 — Model Performance

- Metric cards: Accuracy, Precision, Recall, F1 Score
- Confusion Matrix heatmap (Seaborn)
- Feature Importance bar chart (Logistic Regression coefficients)
- Plain-English explanations of each metric
- Model configuration summary (solver, iterations, scaling, split ratio)

8. Machine Learning Model

8.1 Algorithm

Algorithm: Logistic Regression (Binary Classification)

Logistic Regression fits a sigmoid (S-shaped) function to the input features to model the probability that a student passes. It is interpretable, fast to train, and well-suited for binary classification problems with tabular data.

8.2 Training Configuration

Parameter	Value	Notes
Train/Test Split	80% / 20%	400 training records, 100 testing records
Feature Scaling	StandardScaler	Zero mean, unit variance normalisation
Solver	lbfgs	Limited-memory Broyden-Fletcher-Goldfarb-Shanno
Max Iterations	1000	Ensures convergence on this dataset
Random State	42	For reproducibility

8.3 Key Code Snippets

Data Scaling and Model Training

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_sc, y_train)
```

Prediction with Confidence Score

```
inp = scaler.transform([[hours, attendance, prev_marks,
assignments, sleep]])
pred = model.predict(inp)[0]          # Returns 0 (Fail) or 1 (Pass)
prob = model.predict_proba(inp)[0]    # Returns [fail_prob,
pass_prob]
```

9. Model Performance Metrics

Metric	Expected Value	Description
Accuracy	~88 – 93%	Percentage of students correctly classified overall
Precision	~87 – 92%	When model says PASS, how often is it actually correct?
Recall	~90 – 95%	Of all students who passed, how many did the model catch?
F1 Score	~88 – 93%	Harmonic mean of Precision and Recall — balanced single score

9.1 Metric Definitions

- **Accuracy:** $(\text{True Positives} + \text{True Negatives}) / \text{Total Predictions}$
- **Precision:** $\text{True Positives} / (\text{True Positives} + \text{False Positives})$
- **Recall:** $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$
- **F1 Score:** $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

9.2 Confusion Matrix Interpretation

	Predicted PASS	Predicted FAIL
Actual PASS	True Positive (TP) — correct	False Negative (FN) — missed pass
Actual FAIL	False Positive (FP) — wrong alert	True Negative (TN) — correct

10. How to Run the Project

10.1 Prerequisites

- Python 3.10 or higher installed on your computer
- pip (Python package manager) available in terminal
- A terminal / command prompt / VS Code terminal

10.2 Step-by-Step Instructions

1. Install all required libraries (run once):


```
pip install streamlit scikit-learn pandas numpy matplotlib seaborn
```
2. Save the Python file as Student_Result_Prediction_System.py inside your project folder.
3. Open terminal inside that project folder:
 - **Windows:** Right-click folder in File Explorer → Open in Terminal
 - **Mac / Linux:** Open Terminal, then cd into the folder
4. Run the application:


```
python -m streamlit run Student_Result_Prediction_System.py
```
5. Open in browser (launches automatically):


```
http://localhost:8501
```

Note:

Keep the terminal open while using the app. Press Ctrl+C in terminal to stop the server.

11. Project File Structure

```
student_result_prediction/  
Student_Result_Prediction_System.py <- Main application file  
                                     (entire project)  
README.md                           <- Quick start guide (optional)
```

The entire project is contained in a single Python file. There are no additional folders, databases, or configuration files required to run the basic application.

12. Streamlit Caching

Two Streamlit caching decorators are used to make the app fast and efficient:

@st.cache_data

Applied to `generate_data()`. Stores the 500-student DataFrame in memory so data is not regenerated on every page interaction.

@st.cache_resource

Applied to `train_model()`. Stores the trained Logistic Regression model and scaler so the model is not retrained on every interaction.

Without caching, the app would re-run data generation and model training every time a slider is moved, making the app slow and inconsistent.

13. Limitations

- Uses synthetic data generated by NumPy — not connected to a real student database
- Only one ML model (Logistic Regression) is implemented — no comparison with other algorithms
- No user authentication or role-based access control
- App runs locally on localhost — not accessible from other devices without extra setup
- Prediction accuracy depends on the quality and realism of the synthetic data
- No persistent storage — prediction history is lost when the session ends

14. Future Scope

- Connect to a real student database (MySQL / PostgreSQL) instead of synthetic data
- Add multiple ML models (Random Forest, SVM, XGBoost) and compare their accuracy

- Deploy the app online using Streamlit Cloud (share.streamlit.io) for free
- Add student login and role-based access for Teacher / Student / Admin
- Integrate automatic email alerts for at-risk students
- Export prediction reports as downloadable PDF files
- Add a historical predictions dashboard to track student progress over time
- Implement cross-validation for more robust model evaluation

15. Conclusion

The Student Result Prediction System successfully demonstrates the complete Machine Learning development lifecycle — from synthetic data generation and preprocessing, through model training and evaluation, to deployment as a fully interactive web application. Built entirely in Python without any HTML or JavaScript, the project showcases how modern Data Science tools — Streamlit, scikit-learn, Pandas, Matplotlib, and Seaborn — work together to create real-world AI-powered applications.

The application achieves approximately 88–93% accuracy using Logistic Regression, making it a practical and reliable tool for early identification of at-risk students. This project serves as an excellent internship-level portfolio piece and provides a strong foundation for more advanced enhancements.